

POLITECNICO
MILANO 1863

# Ingegneria del Software

Ingegneria Informatica – Prof. San Pietro

Esercitazione:

Errata Corrige: Esercizio TDE 16/09/16

**My Contacts:**
Giuseppe Sorrentino
giuseppe.sorrentino@polimi.it

# Esercizio TdE 16/09/2016

## Esercizio 3

Si consideri la seguente definizione di una classe Person:

```
public class Person {
  private final String name;
  private final int age;
  public Person(final String theName, final int theAge) {
    name = theName;
    age = theAge;
  }
  public int ageDifference(final Person other) {
    return age - other.age;
 }
  public String toString() {
    return String.format("%s - %d", name, age);
  }
...
}
```

Il seguente frammento di codice definisce un ArrayList people di Person e, a mero titolo di esempio, lo inizializza con alcuni valori.

```
final List<Person> people = Arrays.asList(
  new Person("John", 20),
  new Person("Sara", 21),
  new Person("Jane", 21),
  new Person("Greg", 35));
```

Si rammenta qui la definizione in Java 8 del notissimo interfaccia Comnparator:

```
@FunctionalInterface
public interface Comparator<T> {
  // \result<0 se o1 minore di o2, =0 se uguali, altrimenti >0
  int compare(T o1, T o2)
  ...
}
```

Scrivere un frammento di programma Java 8 in stile funzionale che stampa le persone contenute in people in ordine di eta' decrescente (nell'esempio, Greg, Sara, Jane, John).

Si suggerisce di utilizzare l'operazione `sorted()` definita per tutti gli stream, che prevede un argomento funzionale che sia un Comparator. Ad esempio, se la variabile stringCollection è una List di oggetti String (la cui classe fornisce un metodo compareTo), il seguente frammento definisce uno stream ordinato in ordine crescente.

```
stringCollection
    .stream()
    .sorted((a, b) -> b.compareTo(a))
```

# Comparator

A comparator A.CompareTo(B) returns:

> 0 if A is greater than B
< 0 if A is lower than B
= 0 if A is equal to B

**A.CompareTo(B) can be considered as A - B**

Errata Corrige

# Sorted

sorted((A, B) -> A.compareTo(B)) orders as follows:

- If compareTo returns < 0 → **NO SWAP**

A is less than B,  so A comes before B.

- If compareTo returns > 0 → **SWAP**

A is greater than B,  so B comes before A.

Errata Corrige

# Ascending Ordering (default)

**sorted((A, B) -> A.compareTo(B))**

Example: [10, 50, 20, 30]

**Step 1: (10,50)** → 10.CompareTo(50) → 10 – 50 = - 40 → < 0 → **NO SWAP** → [10, 50, 20, 30]

Errata Corrige

# Ascending Ordering (default)

**sorted((A, B) -> A.compareTo(B))**

Example: [10, 50, 20, 30]

**Step 1: (10,50)** → 10.CompareTo(50) → 10 – 50 = - 40 → < 0 → **NO SWAP** → [10, 50, 20, 30]

**Step 2: (50,20)** → 50.CompareTo(20) → 50 – 20 = 30 → > 0 → **SWAP** → [ 10, **20, 50**, 30]

Errata Corrige

# Ascending Ordering (default)

**sorted((A, B) -> A.compareTo(B))**

Example: [10, 50, 20, 30]

**Step 1: (10,50)** → 10.CompareTo(50) → 10 – 50 = - 40 → < 0 → **NO SWAP** → [10, 50, 20, 30]

**Step 2: (50,20)** → 50.CompareTo(20) → 50 – 20 = 30 → > 0 → **SWAP** → [ 10, **20, 50**, 30]

**Step 3: (50,30)** → 50.CompareTo(30) → 50 – 30 = 20 → > 0 → **SWAP** → [ 10, 20, **30, 50**]

Errata Corrige

# Descending Order

**sorted((A, B) -> B.compareTo(A))**

Example: [10, 50, 20, 30]

**Step 1: (10,50) – 50.CompareTo(10)** → 50 – 10 = 40 → > 0 → SWAP → [50,10,20,30]

Errata Corrige

# Descending Order

**sorted((A, B) -> B.compareTo(A))**

Example: [10, 50, 20, 30]

**Step 1: (10,50) – 50.CompareTo(10)** → 50 – 10 = 40 → > 0 → SWAP → [50,10,20,30]

**Step 2: (10,20) – 20.CompareTo(10)** → 20 – 10 = 10 → > 0 → SWAP → [50, 20, 10, 30]

Errata Corrige

# Descending Order

**sorted((A, B) -> B.compareTo(A))**

Example: [10, 50, 20, 30]

**Step 1: (10,50) – 50.CompareTo(10)** → 50 – 10 = 40 → > 0 → SWAP → [50,10,20,30]

**Step 2: (10,20) – 20.CompareTo(10)** → 20 – 10 = 10 → > 0 → SWAP → [50, 20, 10, 30]

**Step 3: (10, 30) – 30.CompareTo(10)** → 30 – 10 = 20 → > 0 → SWAP → [50,20,30,10]

Errata Corrige

# Descending Order

**sorted((A, B) -> B.compareTo(A))**

Example: [10, 50, 20, 30]

**Step 1: (10,50) – 50.CompareTo(10)** → 50 – 10 = 40 → > 0 → SWAP → [50,10,20,30]

**Step 2: (10,20) – 20.CompareTo(10)** → 20 – 10 = 10 → > 0 → SWAP → [50, 20, 10, 30]

**Step 3: (10, 30) – 30.CompareTo(10)** → 30 – 10 = 20 → > 0 → SWAP → [50,20,30,10]

**Step 4: (50, 20) – 20.CompareTo(50)** → 20 – 50 = -30 → < 0 → NO SWAP → [ 50, 20, 30, 10]

Errata Corrige

# Descending Order

**sorted((A, B) -> B.compareTo(A))**

Example: [10, 50, 20, 30]

**Step 1: (10,50) – 50.CompareTo(10)** → 50 – 10 = 40 → > 0 → SWAP → [50,10,20,30]

**Step 2: (10,20) – 20.CompareTo(10)** → 20 – 10 = 10 → > 0 → SWAP → [50, 20, 10, 30]

**Step 3: (10, 30) – 30.CompareTo(10)** → 30 – 10 = 20 → > 0 → SWAP → [50,20,30,10]

**Step 4: (50, 20) – 20.CompareTo(50)** → 20 – 50 = -30 → < 0 → NO SWAP → [ 50, 20, 30, 10]

**Step 5: (20,30) – 30.CompareTo(20)** → 30 – 20 = 10 → > 0 → SWAP → [50, 30, 20, 10]

Errata Corrige

# Descending Order

**sorted((A, B) -> B.compareTo(A))**

Example: [10, 50, 20, 30]

**Step 1: (10,50) – 50.CompareTo(10)** → 50 – 10 = 40 → > 0 → SWAP → [50,10,20,30]

**Step 2: (10,20) – 20.CompareTo(10)** → 20 – 10 = 10 → > 0 → SWAP → [50, 20, 10, 30]

**Step 3: (10, 30) – 30.CompareTo(10)** → 30 – 10 = 20 → > 0 → SWAP → [50,20,30,10]

**Step 4: (50, 20) – 20.CompareTo(50)** → 20 – 50 = -30 → < 0 → NO SWAP → [ 50, 20, 30, 10]

**Step 5: (20,30) – 30.CompareTo(20)** → 30 – 20 = 10 → > 0 → SWAP → [50, 30, 20, 10]

**Step 6: (20, 10) – 10.CompareTo(20)** → 10 – 20 = -10 → < 0 → NO SWAP → [ 50, 30, 20 , 10]

Errata Corrige

# About the Exercise

```
public int ageDifference(final Person other) {
    return age - other.age;
}
```

We are requested **Descending Order.**

P1.AgeDifference(P2) returns

> 0 → P1 is older.

< 0 → P2 is older

= 0 → Same Age

**Thus, P1.AgeDifference(P2) = P1.Age – P2.Age**

Errata Corrige

# About the Exercise

```
public int ageDifference(final Person other) {
    return age - other.age;
}
```

We are requested **Descending Order.**

**AgeDifference follows exactly CompareTo()**
Definition, as it does:

P1.CompareTo(P2) = P1.Age – P2.Age
A.CompareTo(B) = A - B

Errata Corrige

# About the Exercise

```
public int ageDifference(final Person other) {
    return age - other.age;
}
```

We are requested **Descending Order.**

**To Get Descending Order, we must use:**

```
people.stream()
        .sorted(
            (p1,p2) -> p2.ageDifference(p1)
        ).forEach(System.out::println);
```